

GitStack Connect - Project Report

Executive Summary

GitStack Connect is a comprehensive developer intelligence platform that analyzes GitHub and StackOverflow profiles to build a knowledge graph, detect communities, and provide personalized repository recommendations. The system combines social network analysis, machine learning, and natural language processing to deliver actionable insights about developer networks and collaboration patterns.

1. Project Overview

1.1 Problem Statement

- Developers struggle to discover relevant projects matching their skills and interests
- Limited visibility into developer communities and collaboration networks
- Difficulty understanding one's position and influence within the developer ecosystem
- Lack of intelligent matching between developers and projects

1.2 Solution

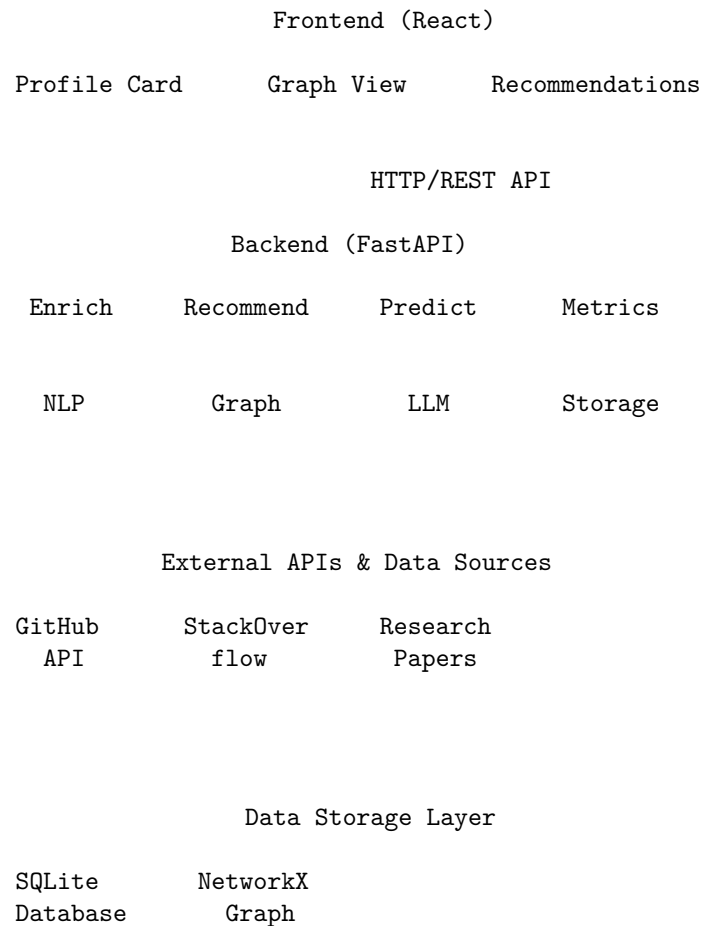
GitStack Connect addresses these challenges by: - **Network Analysis:** Building a knowledge graph from GitHub and StackOverflow data - **Community Detection:** Identifying developer communities using Louvain algorithm - **Intelligent Recommendations:** Providing personalized repository suggestions based on profile analysis - **Role Prediction:** Automatically classifying developer roles using AI and rule-based methods - **Visual Analytics:** Interactive network visualization with centrality metrics - **Project Ideas Search:** Multi-source search across GitHub, StackOverflow, and Dev.to for discovering projects, solutions, and learning resources

1.3 Key Objectives

1. Aggregate developer data from multiple sources (GitHub, StackOverflow)
 2. Build and maintain a knowledge graph representing developer relationships
 3. Detect communities and analyze network structure
 4. Provide intelligent, profile-based recommendations
 5. Visualize network connections and metrics
-

2. System Architecture

2.1 High-Level Architecture



2.2 Component Architecture

Frontend Components

- **App.js:** Main application container with routing and state management
- **ProfileCard.jsx:** Displays user profile, stats, languages, topics, and network metrics
- **GraphView.jsx:** Interactive network visualization using Cytoscape.js
- **TopRecommendations.jsx:** Displays personalized repository recommendations

- **ProjectResults.jsx**: Shows project search results from GitHub, StackOverflow, and research papers

Backend Modules

- **app.py**: FastAPI application with REST endpoints
- **enrich.py**: Core enrichment logic for fetching and processing developer data
- **recommendation.py**: Recommendation engine using graph embeddings and AI
- **graph_ops.py**: Graph algorithms (Node2Vec, centrality, similarity)
- **nlp_ops.py**: NLP operations (topic extraction, summarization, role classification)
- **llm.py**: LLM integration (Groq API) for summaries and ranking
- **storage.py**: Database operations (SQLite)
- **fetchers/**: API clients for GitHub, StackOverflow, and research papers

3. Technology Stack

3.1 Frontend Technologies

Technology	Version	Purpose
React	19.2.0	UI framework
Cytoscape.js	3.33.1	Network graph visualization
Axios	1.13.2	HTTP client for API calls
Lucide React	0.554.0	Icon library
React Scripts	5.0.1	Build tooling

3.2 Backend Technologies

Technology	Version	Purpose
FastAPI	Latest	REST API framework
Uvicorn	Latest	ASGI server
NetworkX	Latest	Graph data structure and algorithms
Node2Vec	Latest	Graph embedding generation
Gensim	Latest	Word2Vec implementation for Node2Vec
SQLAlchemy	Latest	ORM for database operations
SQLite	Built-in	Relational database
Sentence Transformers	Latest	NLP embeddings (SBERT)
Scikit-learn	Latest	Machine learning utilities
BeautifulSoup4	Latest	HTML parsing
Groq	Latest	LLM API for AI summaries and ranking

3.3 External APIs

- **GitHub API:** User profiles, repositories, commits, contributors
 - **StackOverflow API:** User profiles, tags, questions
 - **Research Papers API:** Academic paper search (via dev.to/articles)
-

4. Methodologies and Algorithms

4.1 Knowledge Graph Construction

Graph Structure

- **Nodes:** Users (GitHub, StackOverflow), Repositories, Tags, Topics
- **Edges:**
 - **OWNS:** User $\rightarrow$ Repository
 - **CONTRIBUTES_TO:** User $\rightarrow$ Repository
 - **COLLABORATES_WITH:** User $\rightarrow$ User (via shared repositories)
 - **HAS_TAG:** User/Repo $\rightarrow$ Tag
 - **HAS_PROFILE:** User $\rightarrow$ StackOverflow Profile

Data Aggregation Process

1. **GitHub Data Fetching:**
 - User profile (bio, location, company, followers)
 - Repositories (name, description, language, stars, forks)
 - Commit statistics (frequency, recency, total commits)
 - Collaborators and contributors
 - README content extraction
2. **StackOverflow Data Fetching:**
 - User profile (reputation, badges)
 - Top tags (expertise areas)
 - Question/answer activity
3. **Graph Building:**
 - Normalize node IDs (e.g., `github:username`, `repo:owner/repo`)
 - Create nodes with attributes (type, label, metadata)
 - Create edges with relationship types
 - Store in SQLite database

4.2 Network Analysis Algorithms

4.2.1 Centrality Metrics **Degree Centrality** - Measures number of direct connections - Formula: $C_D(v) = \text{deg}(v) / (n-1)$ - Indicates: Local connectivity and popularity

Betweenness Centrality - Measures how often a node lies on shortest paths - Formula: $C_B(v) = \sum (\text{st}(v) / \text{st})$ for all $s \neq v \neq t$ - Indicates: Bridge/broker role in network

Closeness Centrality - Measures average distance to all other nodes - Formula: $C_C(v) = (n-1) / \sum d(v,t)$ for all t - Indicates: Information spread efficiency

Influence Score - Composite metric combining all centralities - Formula: $(\text{Degree} \times 0.4 + \text{Betweenness} \times 0.4 + \text{Closeness} \times 0.2) \times 100$ - Range: 0-100, higher = more influential

4.2.2 Community Detection Louvain Algorithm - Hierarchical community detection - Optimizes modularity: $Q = (1/2m) \sum [A_{ij} - (k_i k_j / 2m)] (c_i, c_j)$ - Detects communities by maximizing connections within communities - Used to identify developer clusters and highlight same-community members

4.3 Recommendation System

4.3.1 Multi-Stage Recommendation Pipeline Stage 1: Candidate Selection - Filter repositories (exclude user's own repos) - Profile-based filtering: prioritize repos matching user's languages/topics - Score candidates based on:
- Language match: +10 points - Topic match in description: +5 points - Language in description: +3 points - Limit to top 500 candidates

Stage 2: Graph-Based Scoring

Node2Vec Embeddings - Generates 64-dimensional vector representations - Parameters: `walk_length=10`, `num_walks=100`, `dimensions=64` - Captures structural similarity in graph

Cosine Similarity - Measures embedding similarity between user and candidates - Formula: $\cos(\cdot) = (A \cdot B) / (||A|| \times ||B||)$ - Range: -1 to 1, higher = more similar

Jaccard Coefficient - Measures topological similarity - Formula: $J(A,B) = |A \cap B| / |A \cup B|$ - Based on shared neighbors in graph

Score Combination - Formula: $\text{score} = \text{jaccard} \times \text{cosine_sim} + (1 - \text{jaccard}) \times \text{walk_length}$ - Default: `walk\_length = 0.7` (70% weight on embeddings, 30% on topology)

Stage 3: Profile-Based Boost - Additional scoring based on user profile: - Language match: +0.2 - Topic match: +0.15 per topic - Maximum boost: 0.5 - Re-ranks candidates after boost

Stage 4: LLM-Based Ranking - Uses Groq API (Llama 3.1 70B) for final ranking - Input: User profile context + top 20 candidates - Output: Top 5-10 most relevant repositories - Considers: User's bio, languages, topics, AI analysis

4.4 Natural Language Processing

4.4.1 Topic Extraction

- **Method:** SBERT + K-Means Clustering

- **Model:** all-MiniLM-L6-v2 (Sentence Transformers)
- **Process:**
 1. Extract text from READMEs, descriptions, bios
 2. Generate embeddings using SBERT
 3. Cluster embeddings using K-Means (k=3-5)
 4. Extract keywords from cluster centroids
 5. Return top N topics

4.4.2 Summarization

- **Extractive Summarization:** Selects most representative sentences
- **Method:** Centroid-based sentence selection
- **Process:**
 1. Split text into sentences
 2. Generate embeddings for each sentence
 3. Calculate centroid of all embeddings
 4. Select sentences closest to centroid
 5. Generate 5-6 line summary

4.4.3 Role Classification AI-Based Classification (Primary) - Uses Groq API with Llama 3.1 70B - Analyzes: Bio, READMEs, project descriptions, languages - Output: Specific role (e.g., “Frontend Developer”, “ML Engineer”) - Confidence: Based on AI analysis quality

Rule-Based Classification (Fallback) - Keyword-based matching - Categories: - Frontend Developer: React, Vue, Angular, HTML, CSS, JavaScript - Backend Developer: API, server, database, Python, Java - Full Stack Developer: Both frontend and backend keywords - Data Scientist/ML Engineer: Python, TensorFlow, PyTorch, ML, AI - DevOps Engineer: Docker, Kubernetes, CI/CD, infrastructure - Mobile Developer: iOS, Android, Swift, Kotlin - Security Engineer: Security, cryptography, penetration testing - Returns probability distribution over roles

5. Features and Functionality

5.1 User Profile Enrichment

- **GitHub Profile Analysis:**
 - Basic info (name, bio, location, company)
 - Repository statistics (count, stars, forks)
 - Top languages and technologies
 - Activity metrics (commits, frequency)
 - Top worked repositories (based on commit activity)
- **StackOverflow Integration:**
 - Reputation and badge counts

- Top tags and expertise areas
- Community participation metrics
- **AI-Generated Summary:**
 - 5-6 line professional summary
 - Generated using Groq LLM
 - Covers: Role, expertise, contributions, community impact

5.2 Network Visualization

- **Interactive Graph:**
 - Node types: Main User, Repositories, Collaborators, Owners
 - Edge types: Owns, Contributes, Collaborates
 - Color coding: Blue for users, Pink for repos, Orange for StackOverflow
 - Community highlighting: Green borders for same-community members
- **Layout Options:**
 - Force Directed (COSE)
 - Circle
 - Grid
 - Concentric
 - Breadthfirst (Tree)
 - Random
- **Node Type Legend:**
 - Visual guide for different node types
 - Color-coded indicators

5.3 Network Metrics

- **Centrality Measures:**
 - Degree Centrality
 - Betweenness Centrality
 - Closeness Centrality
 - Influence Score (composite)
- **Interactive Tooltips:**
 - Info icons with explanations
 - Describes what each metric measures
 - Explains calculation methods
 - Indicates what the metric tells about the user

5.4 Personalized Recommendations

- **Repository Suggestions:**
 - Based on user profile (languages, topics, bio)
 - Graph-based similarity (Node2Vec embeddings)
 - Topological similarity (Jaccard coefficient)
 - AI-powered final ranking

- Filtered to show only repositories (no users)
- **Profile-Based Filtering:**
 - Prioritizes repos matching user’s tech stack
 - Considers user’s interests and expertise
 - Different recommendations for different users

5.5 Role Prediction

- **Dynamic Role Classification:**
 - AI-based analysis (primary method)
 - Rule-based fallback (when AI unavailable)
 - Probability distribution over roles
 - Confidence scores

5.6 Project Ideas Search

- **User-Initiated Search:** Users can enter any project idea, technology, or topic in the search interface
- **Multi-Source Search:** Simultaneously searches across three platforms:
 - **GitHub Repositories:** Finds relevant code projects, libraries, and repositories
 - * Returns: Repository name, description, language, stars, forks, URL
 - * Use case: Discover existing projects to learn from or contribute to
 - **StackOverflow Questions:** Finds related questions and discussions
 - * Returns: Question title, tags, score, answer status, link
 - * Use case: Find problem-solving approaches and community discussions
 - **Dev.to Articles:** Finds technical articles and tutorials (research papers)
 - * Returns: Article title, author, publication year, description, URL
 - * Use case: Learn about technologies, best practices, and recent trends
- **Features:**
 - **Query-Based Search:** Users enter any search term (e.g., “machine learning”, “React hooks”, “blockchain”)
 - **Independent Pagination:** Each source has its own pagination controls
 - **Real-Time Results:** Fetches results from all three APIs simultaneously
 - **Unified Display:** Results from all sources displayed in organized tabs/sections
- **Workflow:**
 1. User enters a project idea or technology query

2. System searches GitHub, StackOverflow, and Dev.to in parallel
 3. Results are displayed in separate sections for each source
 4. Users can navigate through pages independently for each source
 5. Clicking on any result opens the original source in a new tab
-

6. Data Flow

6.1 User Enrichment Flow

```
User Input (GitHub Username)
↓
Fetch GitHub Profile
↓
Fetch Repositories & READMEs
↓
Fetch Commit Statistics
↓
Fetch Collaborators
↓
Fetch StackOverflow Profile (if available)
↓
Extract Topics (SBERT + K-Means)
↓
Generate AI Summary (Groq LLM)
↓
Classify Role (AI + Rule-based)
↓
Build Graph Nodes & Edges
↓
Store in Database
↓
Return Enriched Profile
```

6.2 Recommendation Flow

```
User Profile
↓
Load Knowledge Graph
↓
Generate Node2Vec Embeddings
↓
Select Candidates (Profile-based filtering)
↓
Compute Similarity Scores
    Cosine Similarity (embeddings)
```

Jaccard Coefficient (topology)
↓
Combine Scores
↓
Apply Profile Boost
↓
LLM-Based Ranking (Groq)
↓
Return Top Recommendations

6.3 Project Ideas Search Flow

User Enters Query (e.g., "machine learning", "React hooks")
↓
Parallel API Calls
 GitHub API → Search Repositories
 StackOverflow API → Search Questions
 Dev.to API → Search Articles
↓
Aggregate Results
 GitHub: Repos with metadata (name, description, language, stars)
 StackOverflow: Questions with tags and scores
 Dev.to: Articles with descriptions and authors
↓
Return Unified Results
↓
Display in Separate Sections with Independent Pagination
↓
User Can Navigate Pages and Click Links to Original Sources

6.4 Metrics Computation Flow

User Node ID
↓
Load Graph from Database
↓
Compute Centrality Metrics
 Degree Centrality
 Betweenness Centrality
 Closeness Centrality
↓
Calculate Influence Score
↓
Detect Communities (Louvain)
↓
Map Community IDs

↓
Return Metrics + Community Info

7. Key Implementation Details

7.1 Database Schema

Nodes Table: - **id:** Primary key (node identifier) - **type:** Node type (user, repo, tag, etc.) - **label:** Human-readable label - **attributes:** JSON blob with metadata - For users: bio, location, company, languages, topics - For repos: description, language, stars, forks - For tags: name, count

Edges Table: - **source_id:** Source node ID - **target_id:** Target node ID - **edge_type:** Relationship type (OWNS, CONTRIBUTES_TO, etc.) - **attributes:** JSON blob with edge metadata (commits, etc.)

7.2 Graph Embeddings

- **Algorithm:** Node2Vec
- **Dimensions:** 64
- **Walk Length:** 10
- **Number of Walks:** 100 per node
- **Caching:** Embeddings computed once and cached
- **Update Strategy:** Recompute when graph structure changes significantly

7.3 API Endpoints

Endpoint	Method	Description
/enrich/{username}	GET	Enrich GitHub user profile
/recommend/{node_id}	GET	Get repository recommendations
/predict/{user_id}	GET	Predict developer role
/metrics/{node_id}	GET	Compute network metrics
/search/project	GET	Search project ideas across GitHub, StackOverflow, and Dev.to

7.4 Error Handling

- **API Failures:** Graceful fallback to rule-based methods
- **Quota Exhaustion:** Detects and prevents repeated API calls
- **Missing Data:** Defaults to generic values (e.g., “Developer” role)
- **Graph Errors:** Returns empty results with error logging

8. Results and Outcomes

8.1 Network Analysis Capabilities

- Successfully builds knowledge graphs from GitHub/StackOverflow data
- Detects developer communities using Louvain algorithm
- Computes centrality metrics for network position analysis
- Visualizes complex network relationships interactively

8.2 Recommendation Quality

- Profile-based filtering ensures relevant suggestions
- Graph embeddings capture structural similarity
- AI ranking provides context-aware recommendations
- Different users receive different, personalized suggestions

8.3 User Experience

- Clean, modern UI with interactive visualizations
 - Comprehensive profile information display
 - Real-time network graph updates
 - Informative tooltips and legends
-

9. Challenges and Solutions

9.1 Challenge: API Rate Limits

Solution: - Implemented polite delays between requests - Caching of computed embeddings - Graceful fallback when APIs are unavailable

9.2 Challenge: Large Graph Performance

Solution: - Limited candidate pool to top 500 - Cached embeddings to avoid recomputation - Efficient graph algorithms (NetworkX optimized)

9.3 Challenge: LLM Quota Exhaustion

Solution: - Implemented quota tracking flag - Fallback to rule-based methods
- Clear error messages to users

9.4 Challenge: Data Quality

Solution: - Multiple data sources (GitHub + StackOverflow) - Data validation and cleaning - Fallback values for missing data

10. Future Improvements

10.1 Short-Term Enhancements

- ☐ Add more data sources (GitLab, Bitbucket)
- ☐ Implement real-time graph updates
- ☐ Add more visualization options
- ☐ Improve recommendation diversity

10.2 Long-Term Enhancements

- ☐ Machine learning model for role prediction
- ☐ Collaborative filtering for recommendations
- ☐ Graph database migration (Neo4j)
- ☐ Real-time collaboration features
- ☐ Mobile application
- ☐ Advanced analytics dashboard

10.3 Scalability Improvements

- ☐ Distributed graph processing
 - ☐ Caching layer (Redis)
 - ☐ Database optimization
 - ☐ API rate limit management
 - ☐ Background job processing
-

11. Conclusion

GitStack Connect successfully demonstrates the power of combining social network analysis, machine learning, and AI to provide intelligent insights into developer ecosystems. The system effectively:

- Aggregates data from multiple sources
- Builds comprehensive knowledge graphs
- Detects communities and analyzes network structure
- Provides personalized, context-aware recommendations
- Visualizes complex relationships intuitively

The project showcases modern web development practices, advanced algorithms, and practical AI integration, making it a valuable tool for developers seeking to understand their position in the developer community and discover relevant projects.

Appendix A: Technical Specifications

A.1 System Requirements

- **Python:** 3.10+
- **Node.js:** 16+
- **Database:** SQLite (can be migrated to PostgreSQL)
- **Memory:** 4GB+ recommended for graph processing
- **Storage:** 500MB+ for database and models

A.2 API Keys Required

- GitHub Token (optional, for higher rate limits)
- Groq API Key (for AI features)
- StackOverflow API (no key required, but rate limited)

A.3 Performance Metrics

- **Enrichment Time:** 5-15 seconds per user
 - **Recommendation Time:** 1-3 seconds
 - **Metrics Computation:** <1 second
 - **Graph Rendering:** Real-time (client-side)
-

Appendix B: References

1. **Node2Vec:** Grover, A., & Leskovec, J. (2016). Node2vec: Scalable feature learning for networks.
 2. **Louvain Algorithm:** Blondel, V. D., et al. (2008). Fast unfolding of communities in large networks.
 3. **Sentence Transformers:** Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks.
 4. **NetworkX:** Hagberg, A., Swart, P., & S Chult, D. (2008). Exploring network structure, dynamics, and function using NetworkX.
-

Document Version: 1.0

Last Updated: 2024

Author: GitStack Connect Development Team